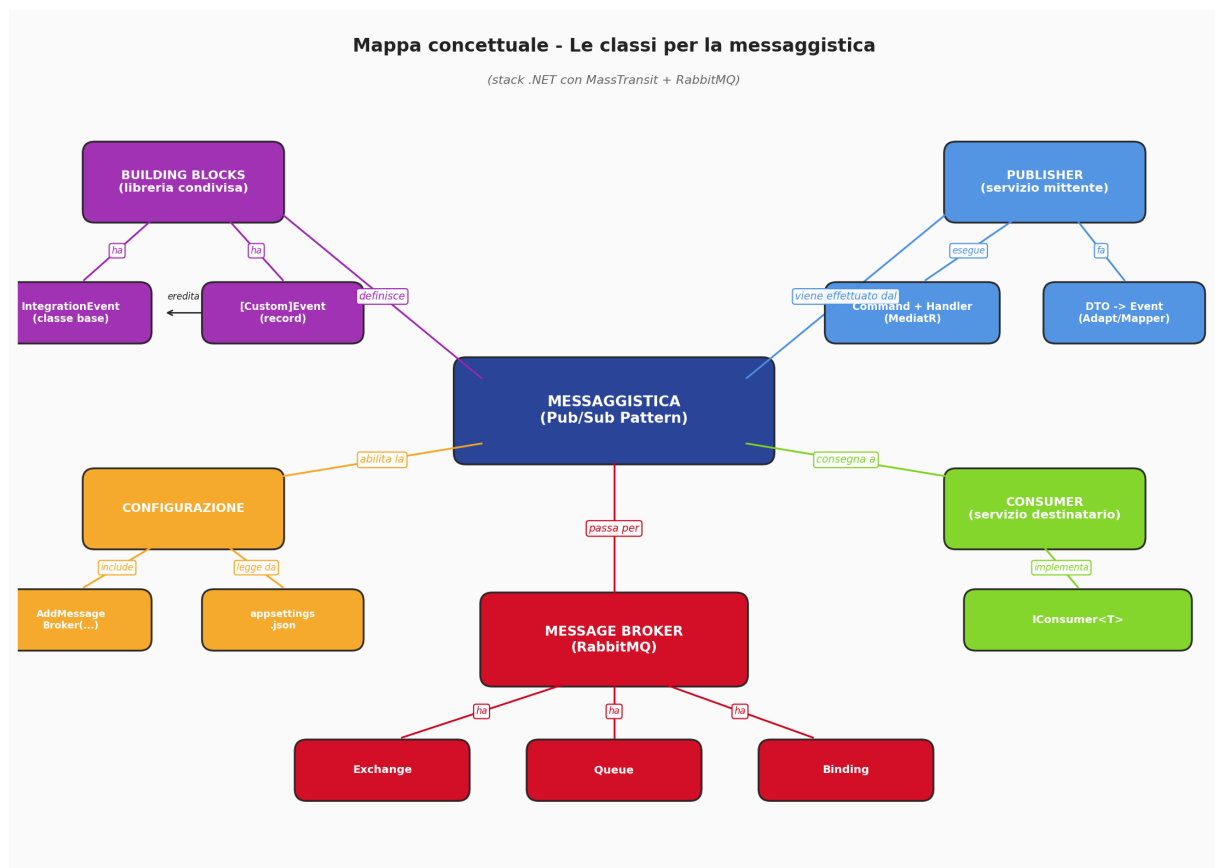


Mappa concettuale

Le classi che devi creare per implementare la messaggistica con un Message Broker



Stack di riferimento

.NET 8 - MassTransit - RabbitMQ

Indice

- 1. Introduzione e legenda della mappa..... 3
- 2. Dove vivono le classi (vista a layer)..... 4
- 3. Gerarchia delle classi (UML)..... 5
- 4. Le 6 classi che devi creare - schede..... 6
 - 4.1 IntegrationEvent (base)..... 6
 - 4.2 [Tuo]Event (specifico)..... 7
 - 4.3 Extension AddMessageBroker..... 8
 - 4.4 Publisher: Command + CommandHandler..... 9
 - 4.5 Consumer: IConsumer..... 10
 - 4.6 (Opzionale) Saga / Process Manager..... 11
- 5. Configurazione (appsettings + Docker + DI)..... 12
- 6. Considerazioni avanzate (retry, DLQ, idempotenza)..... 13
- 7. Ordine consigliato per crearle..... 14
- 8. Checklist finale..... 15

1. Introduzione e legenda della mappa

Questo documento e' una **guida pratica**: ogni volta che dovrai implementare la comunicazione asincrona tra due microservizi tramite un Message Broker, queste sono le classi che dovrai creare. Il riferimento e' lo stack **.NET 8 + MassTransit + RabbitMQ**, ma la mappa concettuale resta valida con qualsiasi tecnologia: i ruoli sono sempre gli stessi.

Cosa NON e' tuo compito creare

Prima di vedere cosa devi fare, mettiamo da parte cio' che *non* devi creare:

- **Il Message Broker stesso** (RabbitMQ): e' un container Docker, non una tua classe. Lo configuri solo.
- **Le primitive AMQP** (Exchange, Queue, Binding): MassTransit le crea automaticamente partendo dai tipi C# che gli passi.
- **La serializzazione** dei messaggi: la fa MassTransit (JSON) senza che tu tocchi nulla.

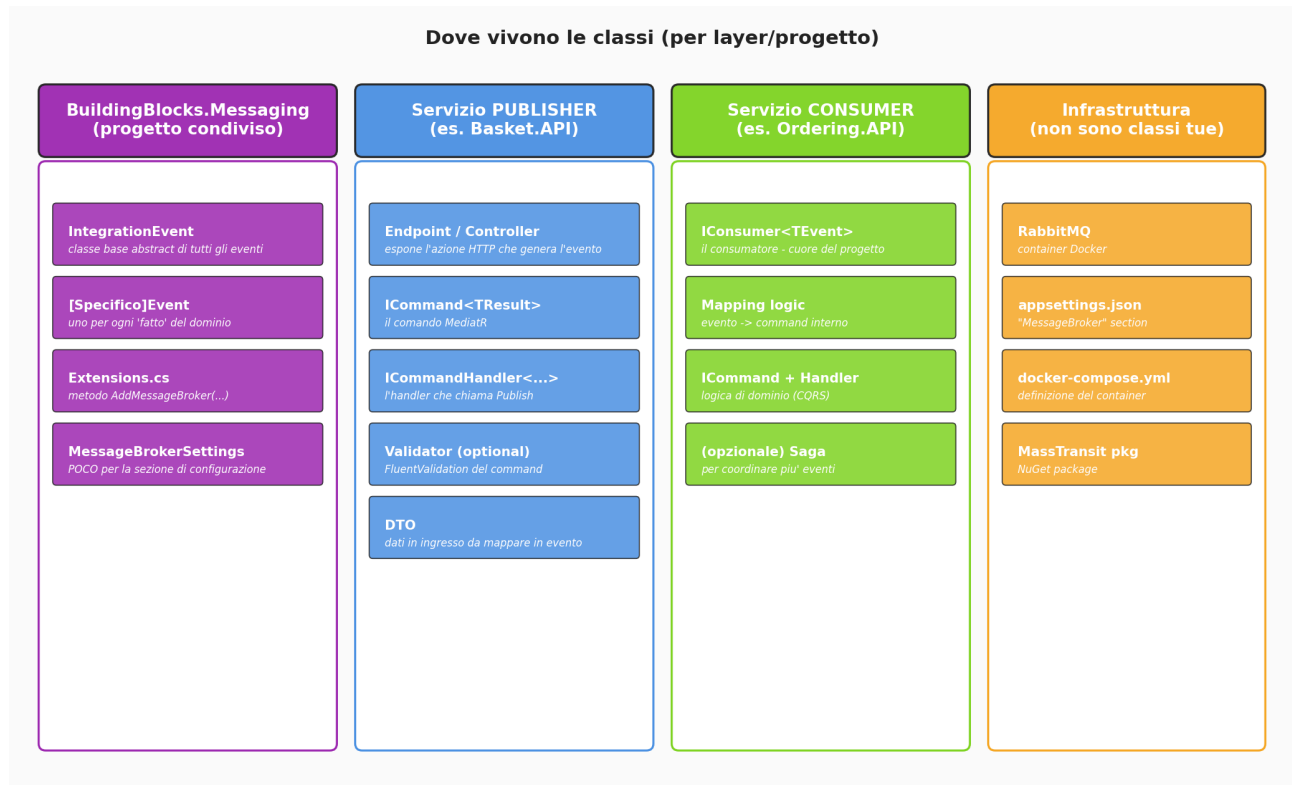
Le 5 famiglie di classi

Guardando la mappa concettuale (sulla copertina), sono 5 i gruppi:

- **BUILDING BLOCKS** - una libreria condivisa con: classe base degli eventi, eventi specifici, metodo di estensione di registrazione.
- **PUBLISHER** - il servizio mittente: endpoint, command, command handler che pubblica tramite `IPublishEndpoint`.
- **BROKER** - infrastruttura, non e' una tua classe.
- **CONSUMER** - il servizio destinatario: classe che implementa `IConsumer<TEvent>`.
- **CONFIGURAZIONE** - non sono classi, sono file: `appsettings.json`, `docker-compose.yml`, registrazione DI.

2. Dove vivono le classi (vista a layer)

Prima di entrare nei dettagli di ciascuna classe, vediamo la mappa a colpo d'occhio: **quale classe vive in quale progetto**. Questa e' anche la buona organizzazione di una solution multi-progetto.



Le tue classi sono distribuite in 3 progetti: una libreria condivisa, un servizio publisher, un servizio consumer.

Regola d'oro: il *contratto* dell'evento (la classe del messaggio) deve stare in un progetto **condiviso**. Sia il publisher sia il consumer lo referenziano. Così parlano la stessa lingua e un cambio di contratto rompe la build subito invece che a runtime.

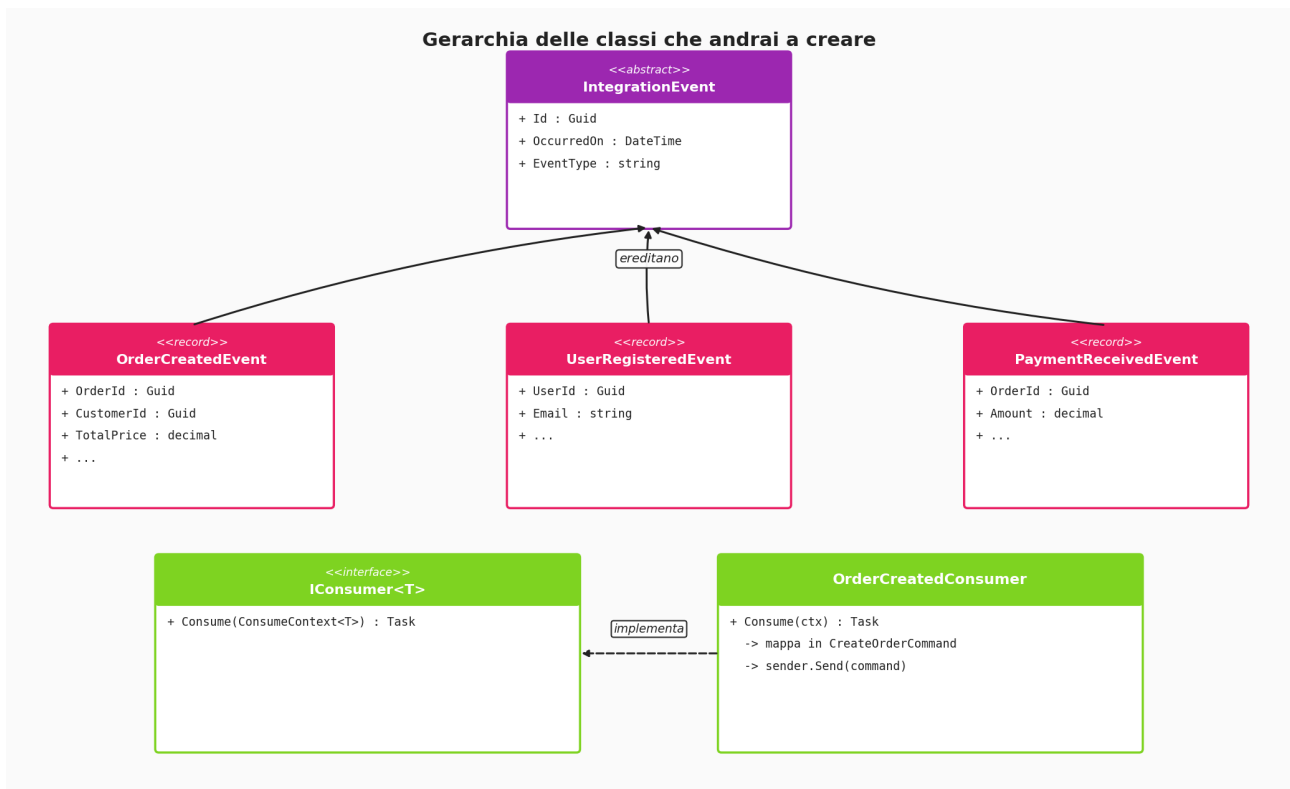
i Perché una libreria condivisa e non NuGet privato?

All'inizio una libreria condivisa nella stessa solution e' la scelta piu' semplice (e' quello che fa eShop con BuildingBlocks.Messaging).

Quando il numero di servizi cresce e i team diventano indipendenti, e' meglio passare a un pacchetto NuGet versionato: ogni servizio decide quando aggiornare il contratto. Un cambio breaking diventa una nuova major version del pacchetto.

3. Gerarchia delle classi (UML)

La gerarchia e' molto piatta: c'e' una classe base astratta (*IntegrationEvent*) da cui ereditano tutti gli eventi concreti, e un'interfaccia (*IConsumer<T>*) che implementi una volta per ciascun evento da consumare.



*Gli eventi ereditano da *IntegrationEvent*. Ogni consumer implementa *IConsumer* per il tipo che ascolta.*

Punti chiave dell'UML:

- Tutti gli eventi **ereditano** da una classe base astratta che fornisce metadati comuni (Id, OccurredOn, EventType).
- Gli eventi sono di solito **record** (immutabili) - in C# 9+ sono perfetti perche' supportano confronti per valore e sintassi compatta.
- Il consumer e' una **classe** (non un record) che **implementa** *IConsumer<T>*. Una classe per evento (o piu' classi per lo stesso evento, se piu' contesti devono reagire).

4. Le 6 classi che devi creare - schede

Per ognuna trovi: ruolo, dove vive, responsabilita', e uno scheletro di codice da copiare e adattare. Sostituisci i placeholder [Tuo] col tuo dominio.

Classe 4.1 IntegrationEvent (classe base)

Ruolo: Classe base *astratta* di tutti gli eventi che attraversano i servizi.

Vive in: BuildingBlocks.Messaging / Events

Responsabilita':

- Fornisce metadati comuni: Id, timestamp, tipo dell'evento.
- NON contiene logica di business: e' solo dati.
- Si crea UNA volta sola per tutta la solution.

```
namespace BuildingBlocks.Messaging.Events;

public abstract record IntegrationEvent
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public DateTime OccurredOn { get; init; } = DateTime.UtcNow;
    public string EventType => GetType().AssemblyQualifiedName!;
}
```

Classe 4.2 [Tuo]Event (evento specifico)

Ruolo: Un evento *concreto* per ciascun fatto del dominio (uno per ogni 'cosa che e' successa' di cui qualcuno potrebbe voler essere informato).

Vive in: BuildingBlocks.Messaging / Events

Responsabilita':

- Eredita da IntegrationEvent.
- Solo **tipi primitivi** nelle property (string, int, Guid, decimal, ...). Niente reference a tipi del dominio del publisher: il consumer non li conosce.
- Nome al participio passato: '...Created', '...Confirmed', '...Cancelled'.
- Una classe nuova per ogni nuovo tipo di evento.

```
namespace BuildingBlocks.Messaging.Events;

public record OrderCreatedEvent : IntegrationEvent
{
    public Guid    OrderId    { get; init; }
    public Guid    CustomerId { get; init; }
    public decimal TotalPrice { get; init; }
    public string  CustomerEmail { get; init; } = default!;
    // ... altri dati primitivi necessari al consumer
}
```

* Best practice: 'self-contained event'

L'evento deve contenere TUTTO quello che serve al consumer per agire. Se OrderCreatedEvent contiene solo l'OrderId, il consumer dovra' chiamare indietro il publisher per leggere i dettagli (= accoppiamento). Inseriscili direttamente nell'evento.

Classe 4.3 Extension method AddMessageBroker

Ruolo: Metodo di estensione che configura MassTransit + RabbitMQ in un solo punto. Lo chiamano sia il publisher sia il consumer.

Vive in: BuildingBlocks.Messaging / MassTransit

Responsabilita':

- Centralizza la configurazione del broker: niente codice duplicato tra servizi.
- Accetta opzionalmente un Assembly per scoprire i consumer.
- Legge host, username, password da appsettings.json.

```
using MassTransit;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using System.Reflection;

namespace BuildingBlocks.Messaging.MassTransit;

public static class Extensions
{
    public static IServiceCollection AddMessageBroker(
        this IServiceCollection services,
        IConfiguration configuration,
        Assembly? assembly = null)
    {
        services.AddMassTransit(cfg =>
        {
            cfg.SetKebabCaseEndpointNameFormatter();

            if (assembly is not null)
                cfg.AddConsumers(assembly);

            cfg.UsingRabbitMq((context, configurator) =>
            {
                configurator.Host(
                    new Uri(configuration["MessageBroker:Host"]),
                    h =>
                    {
                        h.Username(configuration["MessageBroker:Username"]);
                        h.Password(configuration["MessageBroker:Password"]);
                    });
                configurator.ConfigureEndpoints(context);
            });
        });
        return services;
    }
}
```


Classe 4.4 Publisher: Command + CommandHandler

Ruolo: L'handler del command e' il punto in cui il publisher chiama Publish(event).

Vive in: [ServizioPublisher].API / Features

Responsabilita':

- Un ICommand (MediatR) per l'azione (es. CheckoutBasketCommand).
- Un ICommandHandler che inietta IPublishEndpoint.
- Il body costruisce l'evento partendo dal command/DTO e lo pubblica.
- *Trasforma l'azione in un evento di integrazione.*

```
// Command (record)
public record CreateOrderCommand(OrderDto OrderDto)
    : ICommand<CreateOrderResult>;

public record CreateOrderResult(Guid OrderId);

// Handler
public class CreateOrderCommandHandler(
    IOrderRepository repo,
    IPublishEndpoint publishEndpoint) // <-- iniezione
    : ICommandHandler<CreateOrderCommand, CreateOrderResult>
{
    public async Task<CreateOrderResult> Handle(
        CreateOrderCommand cmd, CancellationToken ct)
    {
        // 1. logica di business (es. salva su DB)
        var order = Order.Create(cmd.OrderDto);
        await repo.AddAsync(order, ct);

        // 2. costruisce e PUBBLICA l'evento
        var integrationEvent = new OrderCreatedEvent
        {
            OrderId      = order.Id,
            CustomerId    = order.CustomerId,
            TotalPrice    = order.Total,
            CustomerEmail = order.CustomerEmail
        };
        await publishEndpoint.Publish(integrationEvent, ct);

        return new CreateOrderResult(order.Id);
    }
}
```

! Pattern Outbox (per progetti seri)

C'e' un rischio sottile: se il SaveChanges del DB va a buon fine ma il Publish sul broker fallisce (network, broker giu'), il sistema rimane inconsistente. La soluzione robusta e' il **Transactional Outbox**: salvare l'evento in una tabella 'outbox' nella stessa transazione del DB, e poi un job dedicato pubblica gli eventi della tabella sul broker. MassTransit ha `AddEntityFrameworkOutbox()` che lo gestisce per te.

Classe 4.5 Consumer: IConsumer

Ruolo: La classe che *reagisce* a un evento. E' il cuore della logica del consumer.

Vive in: [ServizioConsumer].Application / Orders / EventHandlers / Integration

Responsabilita':

- Implementa IConsumer<TEvent> di MassTransit.
- Il metodo Consume(ConsumeContext<TEvent> ctx) viene invocato quando un messaggio arriva.
- Tipicamente: mappa l'evento in un command interno e lo invia via MediatR.
- Una classe per ogni evento da consumare.

```
using MassTransit;
using BuildingBlocks.Messaging.Events;
using MediatR;

public class OrderCreatedEventHandler(
    ISender sender,
    ILogger<OrderCreatedEventHandler> logger)
    : IConsumer<OrderCreatedEvent>
{
    public async Task Consume(ConsumeContext<OrderCreatedEvent> ctx)
    {
        logger.LogInformation(
            "Integration Event Handled: {EventType} - {OrderId}",
            ctx.Message.GetType().Name, ctx.Message.OrderId);

        // 1) Mappa evento -> command interno
        var command = MapToCommand(ctx.Message);

        // 2) Inviata alla logica di dominio
        await sender.Send(command, ctx.CancellationToken);
    }

    private static SendNotificationCommand MapToCommand(OrderCreatedEvent e)
        => new(e.CustomerEmail, e.OrderId, e.TotalPrice);
}
```

* Idempotenza nel Consumer

Lo stesso messaggio puo' arrivare **piu' volte** (in caso di retry). Il consumer deve poter essere rieseguito senza creare duplicati: tipicamente memorizzi ctx.MessageId in una tabella 'processed_messages' e scarti i duplicati.

Classe 4.6 (Opzionale) Saga / Process Manager

Ruolo: Quando un processo coinvolge piu' eventi correlati nel tempo, una saga ne orchestra il flusso e mantiene lo stato.

Vive in: [ServizioConsumer].Application / Sagas

Responsabilita':

- Eredita da `MassTransitStateMachine<TState>`.
- Definisce stati (es. *AwaitingPayment*, *Confirmed*, *Cancelled*).
- Reagisce a piu' eventi e fa transitare lo stato.
- Lo stato viene persistito (in memory, EF, MongoDB, Redis...).

```
// Stato persistito
public class OrderSagaState : SagaStateMachineInstance
{
    public Guid CorrelationId { get; set; }
    public string CurrentState { get; set; } = default!;
    public Guid OrderId { get; set; }
    public DateTime CreatedOn { get; set; }
}

// State machine
public class OrderSaga : MassTransitStateMachine<OrderSagaState>
{
    public State AwaitingPayment { get; private set; } = default!;
    public State Confirmed { get; private set; } = default!;

    public Event<OrderCreatedEvent> Created { get; private set; } = default!;
    public Event<PaymentReceivedEvent> Paid { get; private set; } = default!;

    public OrderSaga()
    {
        InstanceState(x => x.CurrentState);

        Initially(
            When(Created)
                .Then(ctx => ctx.Saga.OrderId = ctx.Message.OrderId)
                .TransitionTo(AwaitingPayment));

        During(AwaitingPayment,
            When(Paid).TransitionTo(Confirmed).Finalize());
    }
}
```

Le saghe sono utili per scenari come: *'al checkout, attendi il pagamento entro 10 minuti, altrimenti cancella l'ordine e ripristina il magazzino'*. Per la stragrande maggioranza dei casi, un semplice `IConsumer` e' sufficiente - non introdurre saghe finche' non ne hai bisogno.

5. Configurazione (appsettings + Docker + DI)

Tre file accompagnano sempre la creazione delle classi. Non sono codice C# ma vanno scritti con la stessa cura.

5.1 appsettings.json

In ogni servizio (publisher e consumer) aggiungi la sezione MessageBroker:

```
{
  "MessageBroker": {
    "Host": "amqp://localhost:5672",
    "Username": "guest",
    "Password": "guest"
  }
}
```

5.2 docker-compose.yml

Nel docker-compose della solution aggiungi un servizio per RabbitMQ:

```
messagebroker:
  image: rabbitmq:management      # versione con UI di gestione
  container_name: messagebroker
  hostname: ecommerce-mq         # importante per AMQP
  restart: always
  environment:
    RABBITMQ_DEFAULT_USER: "guest"
    RABBITMQ_DEFAULT_PASS: "guest"
  ports:
    - "5672:5672"                # protocollo AMQP
    - "15672:15672"              # UI di management web
```

5.3 Registrazione DI nel Program.cs / DependencyInjection.cs

Lato publisher (non ha consumer, quindi assembly non passato):

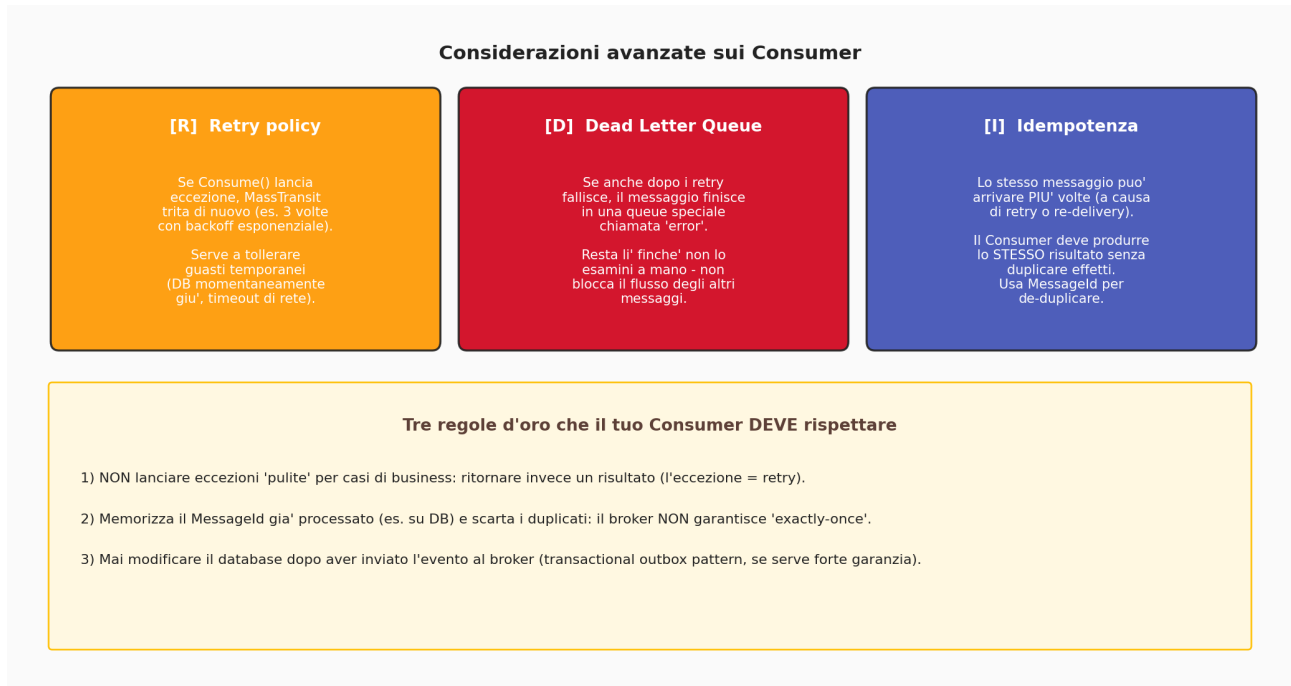
```
// Program.cs del publisher
builder.Services.AddMessageBroker(builder.Configuration);
```

Lato consumer (passa l'assembly che contiene le classi IConsumer):

```
// Es. dentro Ordering.Application/DependencyInjection.cs
services.AddMessageBroker(
  configuration,
  Assembly.GetExecutingAssembly()); // qui ci sono i consumer
```

6. Considerazioni avanzate (retry, DLQ, idempotenza)

Una volta che il flusso base funziona, ci sono tre 'guardie' da aggiungere perche' il sistema sia robusto in produzione.



Retry, Dead Letter Queue e idempotenza: tre concetti indipendenti ma complementari.

Retry policy in MassTransit

Aggiungi nel UsingRabbitMq(...):

```
configurator.UseMessageRetry(r => r.Exponential(
    retryLimit: 3,
    minInterval: TimeSpan.FromSeconds(1),
    maxInterval: TimeSpan.FromSeconds(30),
    intervalDelta: TimeSpan.FromSeconds(2)));
```

Dead Letter Queue

MassTransit crea automaticamente una queue chiamata nome-coda_error dove finiscono i messaggi che hanno esaurito i retry. Tu non devi creare nulla, ma **devi monitorarla** (un alert quando ha messaggi).

Idempotenza

Il broker garantisce *at-least-once* delivery, mai *exactly-once*. Il tuo Consumer deve essere idempotente. Pattern tipico:

```
public async Task Consume(ConsumeContext<OrderCreatedEvent> ctx)
{
    // 1) Controlla se hai già processato questo MessageId
    if (await repo.IsAlreadyProcessedAsync(ctx.MessageId!.Value))
    {
        logger.LogInformation("Skip duplicato {Id}", ctx.MessageId);
        return;
    }

    // 2) Processa
    await DoTheWork(ctx.Message);

    // 3) Marca come processato (stessa transazione di DoTheWork)
    await repo.MarkProcessedAsync(ctx.MessageId!.Value);
}
```

7. Ordine consigliato per crearle

Quando inizi un nuovo progetto e devi implementare il pattern messaggistica da zero, segui questo ordine. Ad ogni passo puoi compilare/testare e procedere solo quando lo step e' verde.



Ordine ottimale: dal generale al particolare, dal contratto all'implementazione.

Dettaglio dei 7 step

- **1. Crea il progetto BuildingBlocks.Messaging** (class library .NET, aggiunge il package `MassTransit.RabbitMQ`).
- **2. Definisci IntegrationEvent** (classe base astratta). E' poche righe, ma e' fondante.
- **3. Scrivi AddMessageBroker** nell'extension. A questo punto il progetto compila ma non c'e' nessun evento.
- **4. Crea il primo evento concreto** ([Tuo]Event). Niente logica, solo il contratto.
- **5. Nel servizio publisher:** il command, l'handler, e la chiamata a `Publish`. Test: il messaggio appare in RabbitMQ Management UI sulla porta 15672.
- **6. Nel servizio consumer:** la classe che implementa `IConsumer<TEvent>`. Test: dopo un publish, il consumer logga l'evento ricevuto.
- **7. Configurazioni finali:** appsettings, docker-compose con il container RabbitMQ, registrazione DI nei due servizi.

8. Checklist finale

Quando avrai finito di implementare la messaggistica, dovresti essere in grado di spuntare ogni voce di questa checklist:

Building Blocks (libreria condivisa)

- ☐ Esiste un progetto class library con i NuGet di MassTransit
- ☐ Esiste una classe `IntegrationEvent` astratta con `Id/OccurredOn/EventType`
- ☐ Esiste almeno un evento concreto che eredita da `IntegrationEvent`
- ☐ Esiste un metodo di estensione `AddMessageBroker`
- ☐ Tutti i tipi nei record evento sono primitivi/serializzabili

Publisher

- ☐ Il servizio referencia `BuildingBlocks.Messaging`
- ☐ Il `Program.cs/DependencyInjection.cs` chiama `AddMessageBroker`
- ☐ L'`appsettings.json` ha la sezione `MessageBroker`
- ☐ Esiste un `Command + Handler` che chiama `publishEndpoint.Publish(...)`
- ☐ Il `Publish` e' chiamato DOPO che lo stato locale e' stato salvato (idealmente outbox)

Consumer

- ☐ Il servizio referencia `BuildingBlocks.Messaging`
- ☐ `AddMessageBroker` e' chiamato passando l'`Assembly` dei consumer
- ☐ Esiste una classe che implementa `IConsumer<TEvent>`
- ☐ Il consumer e' idempotente (gestione `MessageId` duplicati)
- ☐ Il consumer ha un `retry policy` ragionevole configurato

Infrastruttura

- ☐ Il `docker-compose.yml` ha il servizio `messagebroker` con porte 5672 e 15672
- ☐ Riesci ad aprire `http://localhost:15672` e vedere `queue/exchange` creati
- ☐ Hai `monitoring/alerting` sulla DLQ (queue terminanti in `_error`)

= Punto chiave da non dimenticare

Le classi sono solo 6 (5 + 1 opzionale), ma il pattern e' potente. Se le metti in piedi una volta sola con cura, ogni nuovo evento futuro richiede solo: **1 nuovo record evento** nel `BuildingBlocks`, **1 chiamata a Publish** nel publisher, **1 nuova classe Consumer** nel consumer. Tutto il resto e' gia' fatto.

Fine del documento - Buon coding, Hassan!